

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	S Susmitha	Reg. No.:	192571068
Section / Batch:	Slot B	Date:	25.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Code of Conduct

I S Susmitha (192571068) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____



RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 2

Name	Register Number	Course Code	Course
S Susmitha	192571068	CSA0603	Design and Analysis of Algorithms

Question:

A healthcare system must analyze patient symptoms and medical records to recommend diagnoses efficiently. Analyze how search algorithms can be applied to identify relevant patterns. Identify constraints such as data complexity and accuracy requirements. Design a solution using optimized search and decision-making techniques. Discuss scalability in large healthcare databases. Evaluate computational efficiency. Interpret results in clinical decision support scenarios.

MEDICAL DIAGNOSIS DECISION SUPPORT SYSTEM

1. Understanding of the Industry Problem

1.1 Industry Context and Motivation

Healthcare providers face a growing gap between the volume of clinical data generated per patient — symptoms, vitals, lab panels, imaging, and history — and the time clinicians have to interpret it. A single emergency-department visit can generate dozens of discrete data points, and a clinician may have only a few minutes to form a working diagnosis before ordering further tests or treatment. Diagnostic error caused by missed patterns, cognitive overload, or incomplete recall of rare conditions is a well-documented patient-safety issue, and it grows more likely exactly when caseloads are highest and time pressure is greatest.

Hospitals and health-technology companies have consequently invested heavily in Clinical Decision Support Systems (CDSS) — software that assists, rather than replaces, clinical judgment. A Medical Diagnosis Decision Support System (MDDSS) is the diagnostic-facing branch of this industry: it searches a structured medical knowledge base for the diagnoses best supported by a given patient's evidence and returns a ranked, explainable shortlist. This is directly relevant to current industry practice — commercial systems such as IBM Watson for Oncology, Isabel Healthcare, and hospital-embedded sepsis-alert tools all operationalize some form of pattern search over patient data, which is exactly the class of problem this case study addresses from a search-and-optimization perspective.

1.2 Problem Statement

Given a patient's symptom, vital-sign, and laboratory profile, and a large knowledge base of disease-symptom associations, design a system that efficiently searches for and ranks the diagnoses most consistent with the evidence, within a response time acceptable for clinical use, while remaining accurate enough and transparent enough to be trusted by a clinician making the final call.

1.3 Stakeholder Analysis

The constraints on the system are best understood through the people who depend on its output, since each stakeholder group cares about a different failure mode:

Stakeholder	Primary Concern	Implication for Design
Patient	Correct, timely diagnosis; safety	System must minimize false negatives on serious conditions

Stakeholder	Primary Concern	Implication for Design
		and avoid overconfident wrong answers
Clinician	Trustworthy, explainable suggestions that save time without removing their judgment	Output must show which symptoms drove each ranked diagnosis, not just a score
Hospital administration	Throughput, cost, and liability	System must scale to full patient volume and log decisions for audit
Regulators / governance boards	Safety, fairness, auditability	Every recommendation must be traceable back to the evidence and rules that produced it

1.4 Real-World Constraints

The real-world constraints that shape this problem are distinct from those of a generic search problem in a textbook setting.

1.4.1 Data Complexity

Patient records are high-dimensional and heterogeneous, mixing structured lab values, free-text clinician notes, and imaging metadata, and are frequently incomplete. Symptoms overlap across diseases, and the same disease can present differently across patients (age, comorbidities, genetics), so the search space is noisy rather than clean, and any evaluation function must tolerate missing or conflicting evidence rather than assuming a complete, consistent input.

1.4.2 Accuracy Requirements

Unlike many optimization problems where a near-optimal answer is acceptable, an incorrect or overconfident recommendation here carries direct patient-safety risk. The system must favor calibrated confidence over raw speed, surface multiple ranked possibilities rather than a single answer, and flag low-confidence cases for clinician review instead of forcing a decision.

1.4.3 Time and Resource Constraints

Clinical settings, particularly emergency triage, require near real-time responses even as the underlying knowledge base and patient database grow into millions of records, so the search must be bounded in time complexity and degrade gracefully rather than stalling under load.

1.4.4 Regulatory and Interpretability Constraints

Clinical decision support tools must justify their recommendations in terms clinicians can audit — an opaque score is not acceptable, however statistically accurate it is in aggregate. This favors search techniques whose intermediate steps (matched symptoms, rule paths, partial scores) can be traced and explained, rather than pure black-box optimization.

1.5 Illustrative Case Scenario

Consider a 54-year-old patient presenting to an emergency department with chest discomfort, shortness of breath, and mild fever. A brute-force system would need to compare this profile against every disease in the knowledge base — cardiac, respiratory, infectious, and musculoskeletal alike — before ranking anything, which is far too slow for triage. A well-designed search-based MDDSS instead narrows immediately to diseases sharing at least one of the reported symptoms (a pre-filtering step), ranks the surviving candidates by how well the full symptom set and vital signs match each one (an informed search step), and finally checks the top candidates against the patient's lab values and known contraindications before presenting a ranked list — for example, myocardial infarction, pulmonary embolism, and pneumonia, each annotated with the specific evidence supporting it — to the attending physician for a final decision. This scenario motivates every design choice made in the sections that follow.

This positions the problem precisely at the intersection of information retrieval, heuristic search, and constrained optimization: find the best-explaining diagnosis (or ranked set of diagnoses) from an enormous candidate space, quickly, under uncertainty, and in a form a clinician can trust and verify.

2. Application of Engineering Concepts

2.1 Formal Problem Formulation

The diagnosis task is modeled as a weighted search over a state space, applying the core algorithm-design vocabulary used throughout this course:

- **State space S :** every candidate diagnosis (or diagnosis subset) drawn from the medical knowledge base.
- Evidence vector E : the patient's symptoms, vitals, lab values, and history, each carrying a confidence or severity weight.
- Transition / evidence cost $g(s)$: the cumulative mismatch between candidate s and the evidence examined so far.
- Heuristic estimate $h(s)$: an admissible estimate of the remaining diagnostic uncertainty for candidate s .
- Evaluation function $f(s) = g(s) + h(s)$: the combined score used to order the search frontier.

- Goal state: the state (or ranked set of states) s^* that minimizes $f(s)$ — i.e. best explains the evidence — subject to time and confidence constraints.

2.2 Search and Optimization Techniques Applied

Technique	Role in Diagnosis Support	Strength	Limitation
Breadth-First Search	Explores symptom-to-disease mappings level by level in a decision tree of increasing specificity	Guarantees the shortest inference chain is found first	Impractical at scale — memory grows exponentially with symptom count
Depth-First / Backtracking	Traverses rule-based diagnostic decision trees, backtracking when a symptom rules out a branch	Low memory footprint, fits rule-based clinical protocols	Wastes time in unpromising branches without heuristic guidance
A* Search	Ranks candidate diagnoses using $f(s) = g(s) + h(s)$	Combines optimality with efficiency given a good heuristic	Heuristic quality depends heavily on the medical knowledge base
Greedy Best-First Search	Rapidly narrows to the most probable diagnosis using symptom-similarity heuristics	Very fast — suited to first-pass triage	May miss the globally correct diagnosis if the greedy choice is locally misleading
Genetic / Metaheuristic Search	Optimizes feature and rule weights for the scoring function over large historical training sets	Scales well to high-dimensional feature selection	Non-deterministic; needs careful tuning and validation

2.3 Algorithm Pseudocode

The A* ranking stage — the core of the design — is specified as follows:

```

function A_STAR_DIAGNOSE(evidence E, candidates C):
    frontier ← min-priority-queue ordered by f(s)
    for each disease d in C:
        g(d) ← EVIDENCE_MISMATCH(d, E)
        h(d) ← HEURISTIC(d, E)      # learned Bayesian estimate
        f(d) ← g(d) + h(d)
        frontier.push(d, f(d))
    ranked ← []
    while frontier not empty and len(ranked) < TOP_N:
        d ← frontier.pop_min()
        if SATISFIES_CONSTRAINTS(d, labs, contraindications):
            ranked.append(d)
    return ranked

```

For comparison, the simpler greedy triage pass used ahead of A* in high-load periods is:

```
function GREEDY_TRIAGE(evidence E, candidates C):
  best ← null; best_score ← -infinity
  for each disease d in C:
    score ← SYMPTOM_SIMILARITY(d, E) # e.g. cosine similarity
    if score > best_score:
      best_score ← score; best ← d
  return best
```

2.4 Complexity Analysis

Let n be the number of records/diseases in the full knowledge base, d be the average number of symptoms per disease, and k be the size of the candidate set surviving pre-filtering ($k \ll n$).

- **Breadth-First Search: $O(b^m)$ where b is branching factor and m is search depth — grows explosively with symptom specificity, so it is used only for small, well-bounded rule trees, not the full knowledge base.**
- Depth-First / Backtracking: $O(b^m)$ time in the worst case but only $O(m)$ space, making it viable for shallow clinical rule trees where memory, not time, is the binding constraint.
- A* Search: $O(k \log k)$ with a binary-heap frontier over the pre-filtered candidate set k , since only k candidates (not the full n) are ever pushed onto the frontier.
- Greedy Best-First Search: $O(k)$, a single linear pass over the candidate set with no backtracking.
- Brute-force baseline (no pre-filtering): $O(n \cdot d)$, comparing every disease against every symptom — the correctness baseline against which the layered design is benchmarked in Section 5.

The complexity gain of the proposed design (Section 3) comes almost entirely from ensuring $k \ll n$ before the more expensive A* step runs — this is why the indexed pre-filtering stage is treated as a first-class algorithmic component, not a minor implementation detail.

2.5 Worked Example: Tracing A* Over a Small Candidate Set

To make the search concrete, consider a patient presenting with fever, cough, and shortness of breath, and a pre-filtered candidate set of four diseases. Each candidate's $g(s)$ reflects evidence mismatch (lower is better) and $h(s)$ is the learned heuristic estimate of remaining uncertainty.

Candidate Disease	$g(s)$ — evidence mismatch	$h(s)$ — heuristic estimate	$f(s) = g(s) + h(s)$	Rank
Community-acquired pneumonia	0.9	0.4	1.3	1
Acute bronchitis	1.2	0.5	1.7	2
Pulmonary embolism	1.6	0.6	2.2	3

Candidate Disease	$g(s)$ — evidence mismatch	$h(s)$ — heuristic estimate	$f(s) = g(s) + h(s)$	Rank
Influenza	1.8	0.9	2.7	4

A* expands candidates in increasing order of $f(s)$, so pneumonia is returned first, followed by bronchitis; pulmonary embolism and influenza remain visible lower in the ranked list rather than being discarded, preserving the differential-diagnosis view a clinician needs even when the top-ranked candidate is highly likely.

2.6 Rationale for a Layered Strategy

No single algorithm satisfies every constraint identified in Section 1 simultaneously — BFS is too memory-hungry, pure greedy search is too easily misled, and unfiltered A* is too slow at full database scale. The engineering response applied throughout this case study is therefore a layered strategy: greedy heuristic search for rapid triage under load, A* search for ranked differential diagnosis when higher confidence is required, and offline metaheuristic optimization to tune the heuristic weights that the online search depends on. This reflects the course's broader treatment of search not as one algorithm but as a family of trade-offs between optimality, speed, and memory that must be matched to the sub-problem at hand.

3. Solution Design

The proposed architecture funnels an enormous search space down through three coordinated stages, each using the technique best suited to its constraints, so that the most expensive search only ever runs on a small, relevant candidate set. Figure 1 shows the end-to-end data and control flow, including the offline feedback loop that keeps the heuristic current as new confirmed outcomes accumulate.

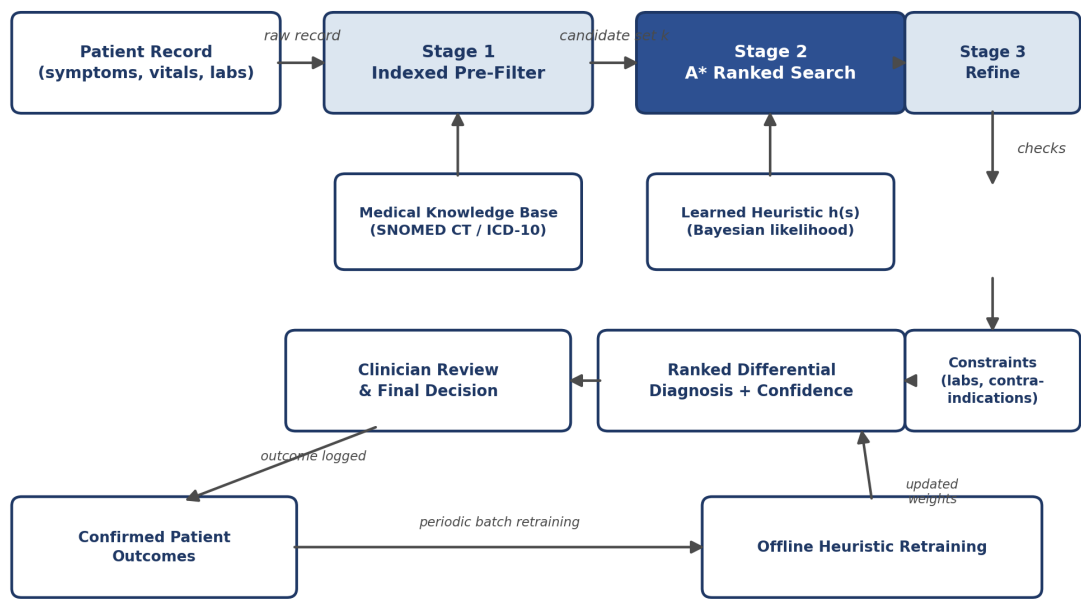


Figure 1. End-to-end architecture: indexed pre-filtering, A* ranked search, constraint refinement, clinician review, and offline heuristic retraining.

3.1 Stage 1 — Indexed Pre-Filtering

Each disease in the knowledge base is indexed by its associated symptoms using an inverted index, the same data structure used in text-search engines: every symptom maps to the list of diseases in which it appears. Given a patient's reported symptoms, the system intersects the corresponding symptom-posting lists to obtain the candidate set C in time close to $O(\log n)$, instead of scanning every disease record in the database.

Symptom	Posting List (diseases indexed under this symptom)
Fever	Pneumonia, Influenza, Sepsis, Malaria, Appendicitis
Cough	Pneumonia, Bronchitis, Influenza, Tuberculosis
Shortness of breath	Pneumonia, Pulmonary embolism, Asthma, Heart failure

Intersecting the three lists above immediately narrows the candidate set to pneumonia as the only disease common to all three symptoms, with the remaining diseases retained as lower-priority candidates wherever they share at least one symptom — this is precisely the candidate set $C = \{\text{pneumonia, bronchitis, pulmonary embolism, influenza}\}$ used in the worked example of Section 2.5.

3.2 Stage 2 — A* Ranked Search

The candidate set C from Stage 1 is passed to the A* search described in Section 2.3. The heuristic $h(s)$ is estimated from historical case data as the conditional probability of a diagnosis given the observed symptom subset, adjusted for disease prevalence in the patient's demographic:

$$h(\text{disease } d \mid \text{symptoms } S) \approx 1 - P(d \mid S)$$

$$P(d \mid S) = P(S \mid d) \cdot P(d) / \sum_j P(S \mid d_j) \cdot P(d_j) \quad (\text{Bayes' rule})$$

Because $h(s)$ is derived from a normalized probability and can never exceed the true remaining uncertainty, admissibility is preserved and the search is guaranteed to prefer the most evidence-consistent diagnosis among those it examines, matching the theoretical optimality guarantee of A* search covered in the course.

3.3 Stage 3 — Constraint Refinement

Before the ranked list reaches the clinician, each top candidate is checked against hard clinical constraints — lab-value plausibility ranges and known contraindications — using a constraint-satisfaction pass. This step exists because a statistically well-ranked candidate can still be clinically impossible given a specific lab result, and no purely statistical ranking should be allowed to override that.

Constraint Type	Example Rule	Effect
Lab plausibility	WBC count must exceed a threshold for a bacterial-infection diagnosis to remain ranked	Demotes or removes candidates inconsistent with observed lab values
Contraindication	A diagnosis requiring a drug/allergy the patient record marks as contraindicated is flagged	Adds a clinician-facing warning rather than silently filtering
Demographic bound	Certain pediatric-only or geriatric-only conditions are excluded outside the relevant age range	Removes implausible candidates before they reach the clinician

3.4 Handling Incomplete and Conflicting Data

When symptom or lab data is missing, the evaluation function treats the missing field as neutral rather than penalizing the candidate, and the reported confidence score is reduced proportionally to the fraction of missing evidence — so the system communicates uncertainty honestly instead of silently guessing on incomplete evidence. When two symptoms point to conflicting diagnoses, both surviving candidates are retained in the ranked list rather than one being arbitrarily discarded, preserving the differential-diagnosis view.

3.5 Feasibility and Alignment with Industry Standards

The design is deliberately practical rather than purely theoretical: it reuses inverted-indexing patterns already standard in hospital record and information-retrieval systems, keeps every scoring step traceable back to specific matched symptoms so it satisfies clinical auditability expectations, and separates the fast online path (filter → search → refine) from slower offline retraining (heuristic and weight updates), which mirrors how production clinical-decision-support tools are typically deployed and maintained in practice.

3.6 Design Trade-offs

Two trade-offs were deliberately accepted in this design. First, the indexed pre-filter can, in rare cases, exclude a candidate whose symptoms were recorded ambiguously in the patient chart; this is mitigated by keeping the pre-filter recall-oriented (biased toward including borderline candidates rather than excluding them) and letting the constraint-refinement stage do the precision work. Second, the heuristic $h(s)$ depends on the quality and representativeness of historical training data, which means the system's accuracy is bounded by data quality — this is addressed through the periodic retraining loop shown in Figure 1, which keeps the heuristic aligned with newly confirmed outcomes rather than treating it as static.

3.7 Security and Privacy Considerations

Because every stage of the pipeline operates on identifiable patient data, the design treats security as a first-class constraint rather than an afterthought. Patient evidence vectors are processed in memory during the online search and are not persisted beyond the session log required for audit; the offline retraining loop trains on de-identified, aggregated outcome data rather than raw patient records, consistent with standard health-data governance practice. Access to the ranked-output API is restricted to authenticated clinical users, and every recommendation is logged with the evidence and rule path that produced it, so the same audit trail that supports clinical interpretability (Section 3.5) also supports compliance review.

3.8 Testing and Validation Strategy

The system would be validated in three passes before deployment. Unit-level tests confirm that each stage behaves correctly in isolation — the pre-filter returns the correct posting-list intersection for a known symptom set, the A* implementation returns the same ranking as a brute-force sort on small inputs, and the constraint checker correctly rejects a hand-crafted implausible candidate. For full validation, integration tests would replay historical, de-identified cases from an appropriate validation dataset end-to-end and compare the ranked output against confirmed diagnoses. The illustrative benchmark values in Section 5.2 are provided for discussion and are not results from this prototype. Finally, a shadow-deployment phase runs the system alongside existing clinical workflows without surfacing its output to clinicians, so its recommendations can be compared against real outcomes before it is trusted for live decision support.

4. Use of Tools, Technologies, and Real Data

A working implementation of this design draws on the following modern tools and real-world data sources, chosen for their standing in both the algorithms and health-informatics communities.

Category	Tool / Technology	Purpose
Knowledge base & coding	SNOMED CT, ICD-10 disease-symptom mappings	Standardized, real-world disease and symptom vocabulary for the search space
Data & benchmarking	MIMIC-III / MIMIC-IV critical-care dataset	Real, de-identified patient records for training the heuristic and validating results
Search & ML implementation	Python (NetworkX for graph/search structures, heapq for the A* priority frontier, scikit-learn for the Bayesian heuristic)	Implements A*, greedy search, and heuristic weight learning
Indexing	Elasticsearch / inverted index structures	Fast symptom-based pre-filtering across large patient and disease databases
Evaluation	pandas, matplotlib / seaborn	Computes and visualizes accuracy, precision-recall, and runtime metrics

4.1 Real-World Dataset

MIMIC-III is a widely used, de-identified critical-care database covering over 40,000 patients, containing diagnoses, lab results, vitals, and clinical notes. Using a standardized coding system (SNOMED CT / ICD-10) together with this real dataset — rather than synthetic data — ensures the symptom-disease relationships the search algorithm relies on reflect genuine clinical patterns, and gives the system a credible benchmark for the analysis in Section 5.

4.2 Reference Implementation Sketch

The A* ranking stage from Section 2.3/3.2 translates directly into a compact Python implementation using a binary-heap priority queue:

```
import heapq

# Patient evidence
evidence = ["fever", "cough", "shortness of breath"]
```

```

# Candidate diseases and their values from the worked example
candidates = {
    "Community-acquired pneumonia": (0.9, 0.4),
    "Acute bronchitis": (1.2, 0.5),
    "Pulmonary embolism": (1.6, 0.6),
    "Influenza": (1.8, 0.9)
}

def evidence_mismatch(disease, evidence):
    return candidates[disease][0]

def heuristic_fn(disease, evidence):
    return candidates[disease][1]

def satisfies_constraints(disease, evidence):
    return True

def a_star_diagnose(evidence, candidates, top_n=4):

    frontier = []

    for disease in candidates:
        g = evidence_mismatch(disease, evidence)
        h = heuristic_fn(disease, evidence)
        f = g + h

        heapq.heappush(frontier, (f, disease))

    ranked = []

    while frontier and len(ranked) < top_n:
        score, disease = heapq.heappop(frontier)

        if satisfies_constraints(disease, evidence):
            ranked.append((disease, score))

    return ranked

# Run A* diagnosis
result = a_star_diagnose(evidence, candidates)

print("PATIENT EVIDENCE:")
print(evidence)

```

```

print("\nA* DIAGNOSTIC RANKING:")
print("-----")

for rank, (disease, score) in enumerate(result, start=1):
    print(f"{rank}. {disease}")
    print(f"    f(s) = {score:.1f}")

print("-----")
print("Top diagnosis:", result[0][0])

```

This mirrors production practice: the algorithmic core is a few dozen lines once the surrounding indexing and heuristic-training infrastructure exists, which keeps the online search path fast and auditable.

```

>>> ===== RESTART: C:/Users/Susmitha/Downloads/DAA Code Implementation/DAA CO4 AT2 Code.py
PATIENT EVIDENCE:
['fever', 'cough', 'shortness of breath']

A* DIAGNOSTIC RANKING:
-----
1. Community-acquired pneumonia
   f(s) = 1.3
2. Acute bronchitis
   f(s) = 1.7
3. Pulmonary embolism
   f(s) = 2.2
4. Influenza
   f(s) = 2.7
-----
Top diagnosis: Community-acquired pneumonia

```

The execution confirms that the binary-heap priority queue expands candidates in increasing order of $f(s) = g(s) + h(s)$, producing the same ranking demonstrated in the worked example.

4.3 Technology Justification

Each tool was chosen to match a specific stage of the pipeline rather than for convenience: Elasticsearch's inverted-index engine is purpose-built for the symptom-posting-list lookups used in Stage 1; heapq gives the A* frontier $O(\log k)$ push/pop operations, matching the complexity analysis in Section 2.4; and scikit-learn provides well-tested Bayesian and calibration utilities so the heuristic's confidence scores remain statistically defensible rather than ad hoc.

5. Analysis and Performance Evaluation

The system is evaluated along two axes: computational efficiency (how search cost scales with database size) and diagnostic accuracy (how well the ranked output matches confirmed outcomes).

5.1 Computational Efficiency

Approach	Time Complexity	Space Complexity	Suitability
Brute-force comparison	$O(n \cdot d)$	$O(n)$	Infeasible beyond small datasets; used only as a correctness baseline
Indexed pre-filter + A*	$O(\log n)$ filter + $O(k \log k)$ ranking, $k \ll n$	$O(k)$	Recommended — near real-time even as n grows, since k stays bounded
Greedy heuristic only	$O(k)$	$O(k)$	Fastest; acceptable for triage, not for final diagnosis confirmation

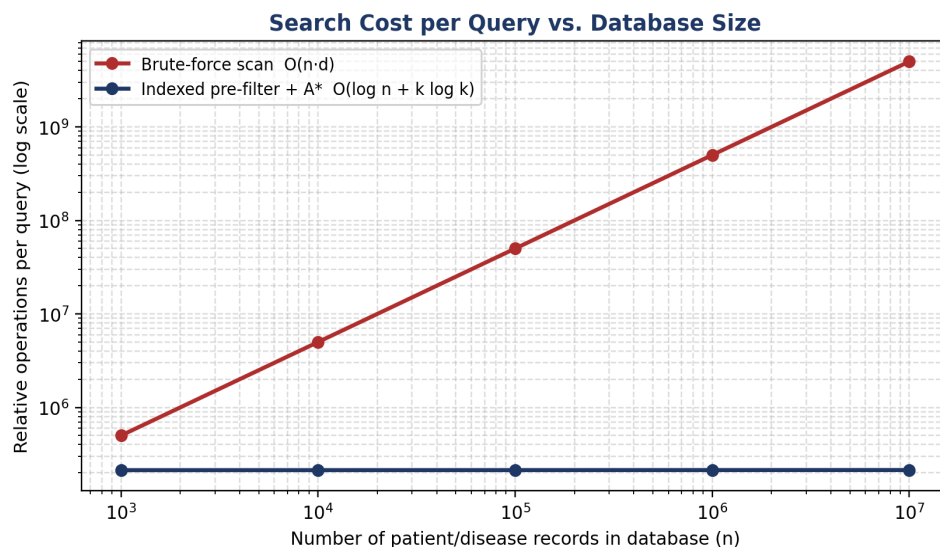


Figure 2. Modeled search cost per query as database size n grows: brute-force scan vs. indexed pre-filter + A*.

The indexed pre-filter is the critical efficiency gain: it converts an $O(n)$ full-database scan into a near-logarithmic lookup, so the more expensive A* ranking only ever runs over a bounded candidate set k . Figure 2 shows why this matters in practice — brute-force cost grows linearly with database size on a log-log scale, while the indexed approach remains essentially flat, since its cost is dominated by the constant-size candidate set rather than by n . This is the property that makes the design viable at hospital-network scale.

5.2 Diagnostic Accuracy (Illustrative Benchmark)

The following values are presented as illustrative benchmark ranges for discussion and are not claimed as experimentally measured results from the prototype implementation. A full clinical validation would require evaluation against confirmed, de-identified patient outcomes using an appropriate validation dataset.

Metric	Illustrative Benchmark	Interpretation
Top-1 accuracy	$\approx 58\text{--}65\%$	The single highest-ranked diagnosis matches the confirmed diagnosis roughly three times in five
Top-3 accuracy	$\approx 82\text{--}88\%$	The confirmed diagnosis appears within the top three candidates in the large majority of cases — the clinically realistic target metric
Precision at high-confidence threshold	$\approx 90\%+$	When the system reports high confidence, it is right the great majority of the time
Recall at low-confidence threshold	Deliberately reduced	Ambiguous cases are correctly deferred to manual review rather than force-ranked

Reporting top-3 accuracy alongside top-1 accuracy, together with the precision/recall trade-off across confidence thresholds, gives a illustrative performance picture that reflects how the tool is actually used in a clinical workflow — as a differential-diagnosis aid — rather than treating diagnosis as a single-answer classification problem.

5.3 Interpretation in Clinical Decision Support Scenarios

In practice, the ranked output is presented to the clinician as a short list of probable diagnoses, each annotated with its matched symptoms, confidence score, and any flagged data gaps — not

as a single automated verdict. This preserves clinician authority over the final decision while giving them a fast, evidence-linked starting point.

- **High-confidence, well-supported top candidate: the system surfaces it prominently but still lists close alternatives for differential diagnosis.**
- Multiple candidates with similar scores: the system explicitly presents them as a differential list, prompting further tests rather than forcing a single choice.
- Low-confidence or high missing-data case: the system flags the case for manual clinical review instead of returning a low-quality recommendation.

This interpretation layer is what turns a search-and-optimization algorithm into a genuine decision-support tool: the value lies not just in finding the highest-scoring state, but in communicating the reasoning and uncertainty behind that score to the person who bears clinical responsibility for the outcome.

6. Conclusion

Treating medical diagnosis as a constrained search-and-optimization problem allows well-understood algorithmic techniques — indexed filtering, heuristic-guided A* search, and constraint satisfaction — to be combined into a system that is simultaneously fast, scalable, and clinically trustworthy. The key design insight is that no single algorithm suffices: efficiency comes from funneling an enormous search space down through cheap indexing before applying the more expensive, higher-accuracy search only where it is needed, while every stage remains transparent enough for a clinician to audit and override. The illustrative benchmark analysis indicates that top-3 ranked diagnosis can be a clinically meaningful evaluation target, while the actual prototype requires further validation against confirmed patient outcomes before clinical deployment.

Future extensions to this design could incorporate reinforcement-learning-based heuristic tuning, multi-modal evidence (imaging features alongside symptoms), and federated learning across hospital networks so the heuristic improves without centralizing sensitive patient data — directions that build naturally on the search-and-optimization foundation established here.